

```
manchodu@Srini:~/OSLAB$ nano m9.c
manchodu@Srini:~/OSLAB$ gcc -o m9 m9.c
manchodu@Srini:~/OSLAB$ ./m9
=====
          PRODUCER-CONSUMER PROBLEM
=====
Buffer size      : 5
Items produced   : 100000

Producer completed successfully.
Consumer completed successfully.
Execution time : 0.602607 seconds
Throughput      : 165945.63 items/second
manchodu@Srini:~/OSLAB$ |
```

```
manchodu@Srini:~/OSLAB$ cat m9.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>

#define BUFFER_SIZE 5
#define ITEMS_TO_PRODUCE 100000

int buffer[BUFFER_SIZE];

int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    int i;

    for (i = 0; i < ITEMS_TO_PRODUCE; i++)
    {
        int item = i + 1;

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}
```

```

        sem_wait(&empty);

        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    int i;

    for (i = 0; i < ITEMS_TO_PRODUCE; i++)
    {
        int item;

        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

```

```

    int i;

    for (i = 0; i < ITEMS_TO_PRODUCE; i++)
    {
        int item;

        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread;
    pthread_t consumer_thread;

    clock_t start, end;
    double execution_time;

    printf("=====\n");
    printf("      PRODUCER-CONSUMER PROBLEM\n");
    printf("=====\n");

    printf("Buffer size      : %d\n", BUFFER_SIZE);
    printf("Items produced   : %d\n", ITEMS_TO_PRODUCE);

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);

    pthread_mutex_init(&mutex, NULL);

    start = clock();

    pthread_create(&producer_thread,
                  NULL,
                  producer,
                  NULL);

```

```

printf("Buffer size      : %d\n", BUFFER_SIZE);
printf("Items produced   : %d\n", ITEMS_TO_PRODUCE);

sem_init(&empty, 0, BUFFER_SIZE);
sem_init(&full, 0, 0);

pthread_mutex_init(&mutex, NULL);

start = clock();

pthread_create(&producer_thread,
              NULL,
              producer,
              NULL);

pthread_create(&consumer_thread,
              NULL,
              consumer,
              NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

end = clock();

execution_time =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("\nProducer completed successfully.\n");
printf("Consumer completed successfully.\n");

printf("Execution time : %.6f seconds\n",
       execution_time);

if (execution_time > 0)
{
    printf("Throughput      : %.2f items/second\n",
          ITEMS_TO_PRODUCE / execution_time);
}

sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);

return 0;
}
manchodu@Srinivas:/OSLAB$ |

```

```
manchodu@Srini:~/OSLAB$ nano m10.c
manchodu@Srini:~/OSLAB$ gcc -o m10 m10.c
manchodu@Srini:~/OSLAB$ ./m10
```

```
=====
DEADLOCK DEMONSTRATION
=====
```

```
Thread 1: Trying to acquire Resource 1...
Thread 1: Acquired Resource 1
Thread 2: Trying to acquire Resource 2...
Thread 2: Acquired Resource 2
Thread 2: Trying to acquire Resource 1...
Thread 1: Trying to acquire Resource 2...
|
```

```

void *thread2(void *arg)
{
    printf("Thread 2: Trying to acquire Resource 2...\n");

    pthread_mutex_lock(&resource2);

    printf("Thread 2: Acquired Resource 2\n");

    sleep(1);

    printf("Thread 2: Trying to acquire Resource 1...\n");

    pthread_mutex_lock(&resource1);

    printf("Thread 2: Acquired Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_mutex_init(&resource1, NULL);
    pthread_mutex_init(&resource2, NULL);

    printf("=====\n");
    printf("        DEADLOCK DEMONSTRATION\n");
    printf("=====\\n\\n");

    pthread_create(&t1, NULL, thread1, NULL);
    pthread_create(&t2, NULL, thread2, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1);
    pthread_mutex_destroy(&resource2);

    return 0;
}

```